Reinforcement Learning-Based Robotic Vacuum Cleaner Path Planning

Coverage planning with Q-learning and SARSA

Corey Chiang | Electrical Engineering, National Chung Hsing University

A Python simulation of a room-cleaning task: visit every reachable dirty cell, avoid furniture, and return to the charging position. The project compares two tabular temporal-difference methods on a fixed bedroom map and records how their learned policies complete the task.

Room illustration



Simulation map

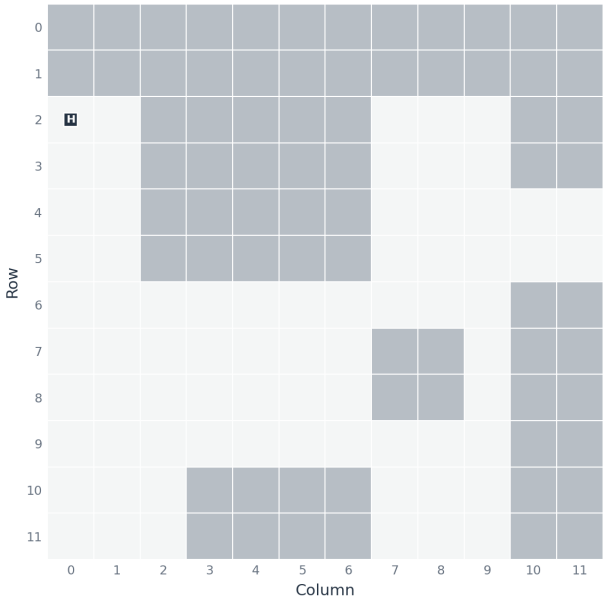


Figure 1. Original portfolio room illustration (left) and the code-defined simulation map (right). Gray cells are blocked; H marks the start and required finish. Obstacle coordinates are specified in code.

12 x 12
72 reachable free cells

71 targets
Home is initially clean

4 actions
East, west, north, south

What this project demonstrates

Environment modeling, reward accounting, tabular control, reproducible experiments, and verification of complete trajectories. The reviewed package separates training, evaluation, and visualization so each reported result can be traced to settings and logs.

Across 10 matched seeds, Q-learning averages **6.4% fewer completion steps** and **22.5% fewer revisit moves**. The clearest difference is in training completion rate (page 4).

Model and learning loop

The state includes both position and cleaning history.

State and termination

Each state is stored as **(position_index, remaining_target_mask)**. A bit marks each dirty target; visiting that target clears its bit. This preserves the original dirty-map information while making the dictionary keys smaller. An episode succeeds only when the mask is zero and the robot is back at home.

Transition event	Reward components	Net reward
Wall or furniture collision	-1 move cost	-1
Move to an already clean cell	-1 move cost	-1
First visit to a dirty target	-1 move + 3 clean	+2
Return home after cleaning all targets	-1 move + 100 finish	+99

Action-value updates

Q-learning: bootstrap from the largest next-state value.

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

SARSA: bootstrap from the next epsilon-greedy action actually sampled.

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma\, Q(s',a') - Q(s,a)]$$

At a true terminal transition, both targets reduce to the immediate reward. A training step cap is an external truncation, so it is logged separately and does not remove the bootstrap term.

Choices retained for source reproducibility

Float32 Q-values, Python's seeded random generator, and linear epsilon decay are retained. Ties choose the first maximum in E/W/N/S order, as in the source. This rule creates a directional bias; it is a compatibility choice. Greedy evaluation uses read-only table lookups.

How the policies differ

Matched settings, seed 42. All 10 seeds are included in the next page's aggregate results.

Method	Episodes	Train cap	Alpha	Gamma	Steps
Q-learning	8,000	500	0.3	0.95	94
SARSA	8,000	500	0.3	0.95	104

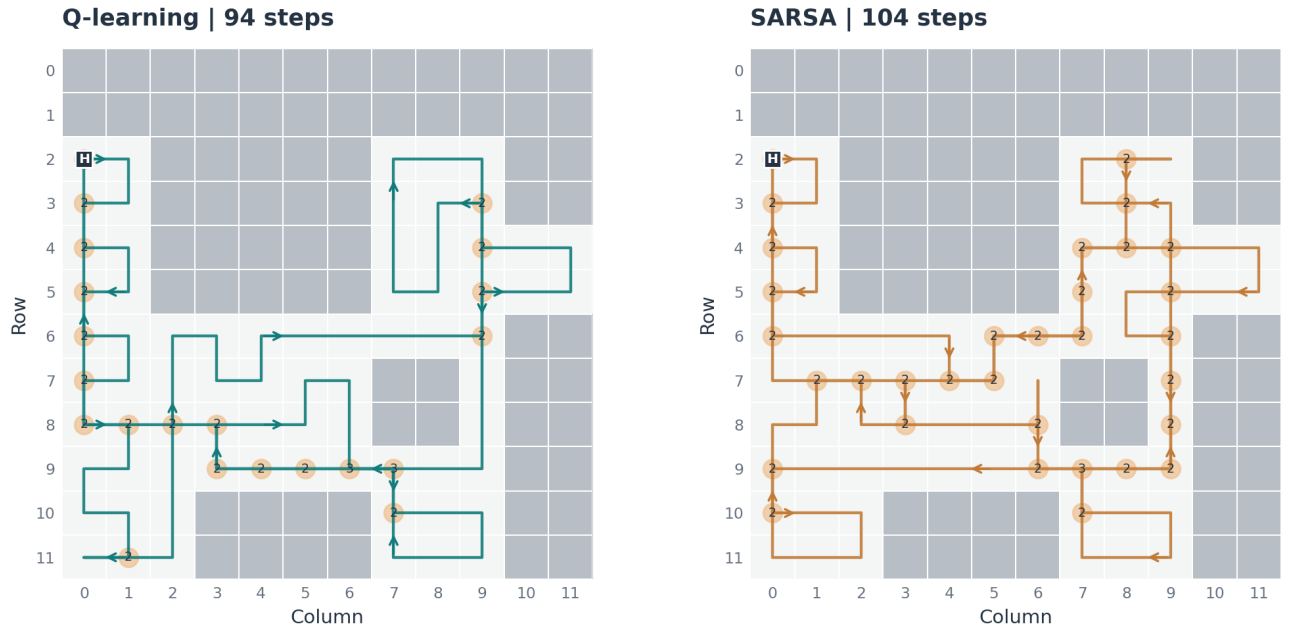


Figure 2. Greedy evaluation trajectories. Each has 100% target coverage, returns home, and records zero collisions. Tan markers identify revisited cells; numbers show visit counts. Q-learning has 23 revisit moves and SARSA has 33, including return home.

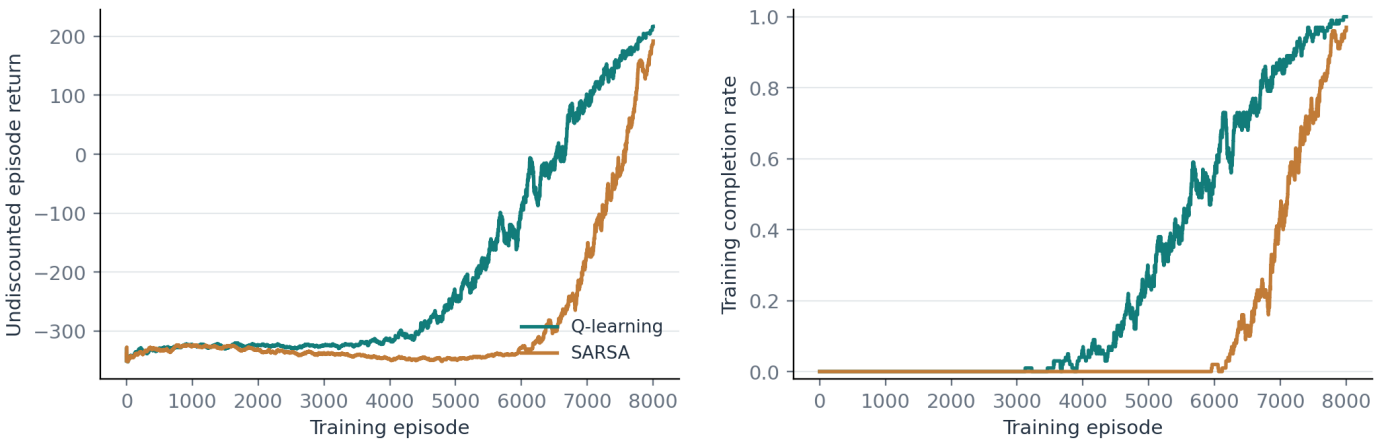


Figure 3. Seed 42 training histories with identical settings, shown as trailing 100-episode means. The included comparison GIF replays both evaluated routes on the same step clock. This single seed illustrates behavior; it is not the aggregate result.

Matched-setting comparison

8,000 episodes; training cap 500; alpha 0.3; gamma 0.95; identical map and schedule.

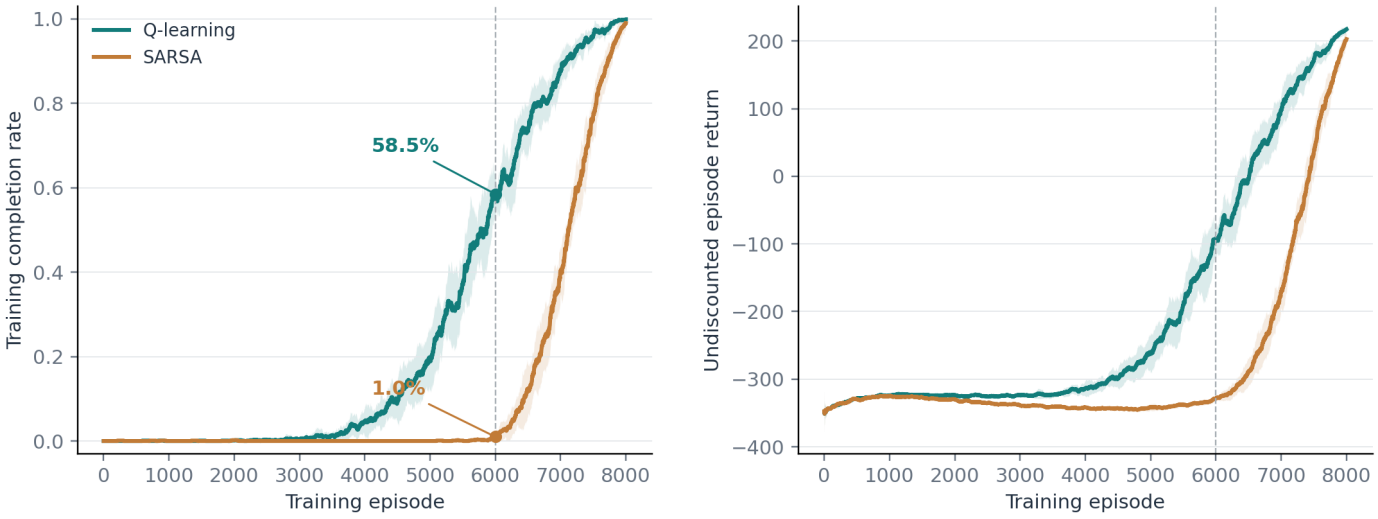


Figure 4. Mean of 100-episode rolling statistics across 10 seeds; shading is one sample standard deviation. At episode 6,000, mean training completion is 58.5% for Q-learning and 1.0% for SARSA. This measures the exploratory training policy, not a greedy checkpoint.

Method	Final steps mean +/- SD	Revisit mean	First 80% window mean episode	Greedy success
Q-learning	93.0 +/- 2.5	22.0	6,502	10 / 10
SARSA	99.4 +/- 6.7	28.4	7,497	10 / 10

Milestone: first full 100-episode window with at least 80% completed training episodes; all runs reach it. Revisit moves include the required return home. Seeds: 1-8, 42, 789.

What the difference means

The largest separation is in learning progress, with a 57.5-percentage-point completion gap at episode 6,000. Final greedy routes differ more modestly: Q-learning averages 6.4% fewer steps and 22.5% fewer revisits. It wins eight paired seeds, ties one, and loses one.

Both learners finish in all 10 final evaluations. A nearest-target BFS baseline takes 102 steps. These results concern one known, deterministic room; they do not establish general navigation performance or physical safety.

Reproduce and inspect

The figures and metrics in this report are backed by the included run records.

Component	Responsibility
environment.py	Map validation, compact state, transitions, reward accounting
learning.py	Configuration, exploration, Q-learning and SARSA updates
evaluation.py	Read-only rollouts, coverage checks, classical baseline
plots.py	Paths, learning curves, and trajectory-based GIF replay
results/ and tests/	Recorded experiments, reference trajectories, behavioral tests

Run from the project root

```
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python -m vacuum_rl --preset portfolio --out runs/portfolio --render --gif
python tools/verify_reference_runs.py --results runs/portfolio
python -m vacuum_rl --preset matched --seeds 1 2 3 4 5 6 7 8 42 789
python -m vacuum_rl --replay results/portfolio/qlearning_seed789
```

Recorded environment: Python 3.12.14, NumPy 2.3.5, Matplotlib 3.10.8, Pillow 12.3.0. Each run includes configuration and version metadata, episode history, a step-by-step rollout, and a replayable trajectory.

Validation and limits

The package passes 14 automated tests. The uploaded portfolio's 84-step Q-learning and 96-step SARSA runs are reproduced action by action under their original, unequal settings. The original report's 94-step Q-learning run matches the shared preset at seed 789.

The 72-step reference is a counting lower bound (71 new targets plus one move home), not a proved optimum. No sensors, motion noise, battery use, or dynamic obstacles are modeled. Compact keys reduce overhead; the tabular state space remains exponential.

Based on the vacuum task from the four-member Group 9 course project. Portfolio prepared by Corey Chiang. The 2026-10-23 review, refactor, and matched-setting experiments are subsequent additions, documented separately from the original submission.

Repository: github.com/corey1218-hub/rl-robotic-vacuum-cleaner